Perfetto! Ti propongo un'esercitazione guidata sul classico **problema del produttore-consumatore**, suddivisa in **tre step progressivi** per mostrare l'importanza della sincronizzazione e delle primitive `wait()` / `notify()` in Java.

🧪 Esercitazione: Produttore-Consumatore con monitor `Product`

✅ Step 1 – Versione senza sincronizzazione

Obiettivo: osservare il comportamento senza alcuna protezione.
Problema: accesso concorrente non controllato → race condition.

🔷 `Product.java` (già fornita)

```java
public class Product {
    private int element;

    public Product(int n){
        this.element = n;
    }

    public void setElement(int element) {
        this.element = element;
    }

    public int getElement(){
        return this.element;
    }
}
```

🔷 `Consumer.java`

```java
public class Consumer extends Thread{

    private Product prod;
    private int times;

    public Consumer(Product prod, String name, int times){
        this.prod = prod;
        this.setName(name);
        this.times = times;
    }

    public void run(){
        try{
            for(int i = 0; i < times; i++ ){
                System.out.println(this.getName() + " dormo...");
                Thread.sleep(5000);
                int tmp = prod.getElement();
                System.out.println(this.getName() + " ho consumato: " + tmp);
                prod.setElement(tmp);
            }
        }
        catch(InterruptedException e){
            System.out.println(this.getName() + " sono stato interrotto ");
        }

    }
}
```

🔷 `Producer.java`

```java
public class Producer extends Thread{

    private Product prod;
    private int times;

    public Producer(Product prod, String name, int times){
        this.prod = prod;
        this.setName(name);
        this.times = times;
    }

    public void run(){
        try{
            for(int i = 0; i < times; i++ ){
                System.out.println(this.getName() + " dormo...");
                Thread.sleep(5000);
                int tmp = (int)Math.round(Math.floor(Math.random() * 10 + 1));
                System.out.println(this.getName() + " ho prodotto: " + tmp);
                prod.setElement(tmp);
            }
        }
        catch(InterruptedException e){
            System.out.println(this.getName() + " sono stato interrotto ");
        }

    }
}
```

🔷 `Main.java`

```java
public class Main {
    public static void main(String[] args) {
        Product p = new Product(0);//monitor, sezione critica
        Consumer c1 = new Consumer(p,"C1", 5);
        Producer p1 = new Producer(p,"P1", 5);
        c1.start(); p1.start();
        try{
            c1.join();
            p1.join();
        }
        catch (InterruptedException e){
            throw new RuntimeException();
        }
    }
}
```

🧠 **Domande guida:**

- Cosa succede quando i thread accedono contemporaneamente al prodotto?
- Noti incoerenze nei valori stampati?

🔒 Step 2 – Sincronizzazione con `synchronized`

Obiettivo: usare il meccanismo base per evitare conflitti tra thread.

🔷 `Product.java` modificata:

```java
public class Product {

    private int element;

    public Product(int n){
        this.element = n;
    }

    public synchronized void setElement(int element) {
```

```
        this.element = element;
    }

    public synchronized int getElement(){
        return this.element;
    }
}
```

🧠 **Domande guida:**

- L'output è più ordinato ora?
- Riesci a vedere il legame causa-effetto tra `produce` e `consume`?

⏱️ Step 3 – Uso di `wait()` / `notify()` con controllo `while`

Obiettivo: sincronizzare la produzione e il consumo in modo corretto.
Concetto chiave: il consumatore deve **aspettare** finché non c'è un prodotto disponibile, e il produttore deve **aspettare** finché il prodotto non è stato consumato.

🔷 `Product.java` (versione monitor):

```
//classe Monitor
public class Product {

    private int element;

    private boolean produced; // variabile guardia

    public Product(int n){

        this.element = n;
        this.produced = false;
    }

    public synchronized void setElement(int element) {
        while(produced){
            try{
                System.out.println(Thread.currentThread().getName() + " aspetto");
                wait();
            }
            catch(InterruptedException e){
                System.out.println(Thread.currentThread().getName() + " sono stato interrotto mentre
aspettavo di poter produrre");
            }
        }

        this.element = element;
        produced = true;
        notifyAll();
    }

    public synchronized int getElement(){

        while(!produced){
            try{
                System.out.println(Thread.currentThread().getName() + " aspetto");
                wait();
            }
            catch(InterruptedException e){
                System.out.println(Thread.currentThread().getName() + " sono stato interrotto mentre
aspettavo di poter consumare");
            }
        }
        produced = false;
        notifyAll();
        return element;
```

```
    }
}
```

🔷 `Producer` e `Consumer` ora usano `while` anziché `for` (migliore flessibilità)

```
public class Producer extends Thread {
    private Product prod;
    private int times;

    public Producer(Product prod, String name, int times){
        this.prod = prod;
        this.setName(name);
        this.times = times;
    }

    public void run(){
        int count = 0;
        while(count < times){
            int value = (int)(Math.random() * 100);
            prod.setElement(value);
            count++;
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                interrupt();
            }
        }
    }
}
```

```
public class Consumer extends Thread {
    private Product prod;
    private int times;

    public Consumer(Product prod, String name, int times){
        this.prod = prod;
        this.setName(name);
        this.times = times;
    }

    public void run(){
        int count = 0;
        while(count < times){
            int value = prod.getElement();
            count++;
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                interrupt();
            }
        }
    }
}
```

🧩 Conclusioni

Questa esercitazione mostra chiaramente:

- Il caos senza sincronizzazione
- L'ordine introdotto da `synchronized`
- L'efficienza e correttezza dell'uso di `wait()` / `notify()`
  → pattern classico per i monitor in Java